

32s left



ALL



1

2

3

1. Remove Integers

Given the first 2^N integers in which K of them are special. We want to remove all the 2^N integers.

There are 3 choices to remove integers, starting with 2^N integers:

1. If the current part contains L integers and X of them are special, the cost of removing all the L integers is $L * X * P$ where P is a constant.
2. If the current part contains L integers and none of them is special, the cost of removing all the L integers is Q , where Q is a constant.
3. If the current part contains L integers, and L is divisible by 2, then we have 2 choices:
 1. Calculate the cost of this part by using method 1 or method 2
 2. Divide this part into two parts containing $L/2$ integers each, then add the costs of those two parts together.

Find the minimum cost of removing all 2^N integers, modulo $(10^9 + 7)$.

NOTE: We cannot reshuffle integers while dividing the current part into two smaller parts. If the current part contains integers [1, 2, 3, 4], the only correct way to divide it is [1, 2] and [3, 4].

Example

$N = 2$

$P = 2$

Language C++

Autocomplete Ready ⓘ

```
9  /*
10  * Complete the 'removeIntegers' function below.
11  *
12  * The function is expected to return an INTEGER.
13  * The function accepts following parameters:
14  * 1. INTEGER N
15  * 2. INTEGER P
16  * 3. INTEGER Q
17  * 4. INTEGER_ARRAY special_integers
18  */
19  int mod=1e9+7;
20  long long getAns(vector<int> &nums, int s, int e,
    unordered_set<int> &set, int p, int q,
    vector<vector<long long>> &dp)
21  {
22      if(s==e) {
23          if(set.find(nums[s])!=set.end()) return
    p%mod;
24          return q%mod;
25      }
26      if(dp[s][e]!=-1) return dp[s][e];
27      long long ans;
28      int spe=0;
29      for(int i=s;i<=e;i++)
30      {
31          if(set.find(nums[i])!=set.end()) spe++;
32      }
```

Line: 9 Col: 1

Test

Custom

Run
Code

Run
Tests

Submit

29s left

Example

 $N = 2$ $P = 2$ $Q = 1$ $special_integers = [1, 3]$

Start with the first 2^N integers: [1, 2, **3**, 4] in which integers 1 and 3 are special, shown in bold.

1 Consider the entire array:

- The length of the segment is 4, there are 2 special integers, and $P = 2$. Using method 1 costs $4 * 2 * 2 = 16$.
- Method 3 can also be used since the length of the segment is divisible by 2.
- The cost of removing all elements at once is 16.

Method 3:

Divide the array into two parts: [1, 2] and [3, 4].

- The part [1, 2] contains 1 special integer.
 - Using method 1 on this part costs $2 * 1 * 2 = 4$.
 - Method 3 can also be used.
- Similarly, the part [3, 4] contains 1 special integer.
 - Using method 1 costs $2 * 1 * 2 = 4$.
 - Method 3 can also be used.
- Hence, we can remove all the integers in $4 + 4 = 8$ with these two segments.

Language C++

Autocomplete Ready ⓘ

```

9  /*
10  * Complete the 'removeIntegers' function below.
11  *
12  * The function is expected to return an INTEGER.
13  * The function accepts following parameters:
14  * 1. INTEGER N
15  * 2. INTEGER P
16  * 3. INTEGER Q
17  * 4. INTEGER_ARRAY special_integers
18  */
19  int mod=1e9+7;
20  long long getAns(vector<int> &nums, int s, int e,
    unordered_set<int> &set, int p, int q,
    vector<vector<long long>> &dp)
21  {
22      if(s==e) {
23          if(set.find(nums[s])!=set.end()) return
p%mod;
24          return q%mod;
25      }
26      if(dp[s][e]!=-1) return dp[s][e];
27      long long ans;
28      int spe=0;
29      for(int i=s;i<=e;i++)
30      {
31          if(set.find(nums[i])!=set.end()) spe++;
32      }

```

Line: 9 Col: 1

Test
ResultsCustom
InputRun
CodeRun
Tests

Submit

27s left



ALL



1

Method 3 again:

Now divide the segments [1, 2] and [3, 4] into two parts each, yielding [1], [2], [3] and [4].

- Each of the parts with a special number costs $1 * 1 * 2 = 2$
- Each of the parts without a special number costs 1.
- The total cost is $2 + 1 + 2 + 1 = 6$.

The minimum cost is 6.

Function Description

Complete the function `removeIntegers` in the editor below.

`removeIntegers` has the following parameters:

`int N`: array values range from 1 to 2^N , inclusive

`int P`: the cost of a special number

`int Q`: the cost of a segment with no special numbers

`int special_integers[K]`: the special numbers

Returns

`int`: the minimum cost to remove all integers, modulo $(10^9 + 7)$.

Constraints

- $1 \leq N \leq 30$
- $1 \leq P, Q \leq 10^4$
- $1 \leq \text{special_integers.size}() K \leq 10^5$
- $1 \leq \text{special_integers}[i] \leq 2^N$

Language C++

Autocomplete Ready ⓘ

```
9  /*
10  * Complete the 'removeIntegers' function below.
11  *
12  * The function is expected to return an INTEGER.
13  * The function accepts following parameters:
14  * 1. INTEGER N
15  * 2. INTEGER P
16  * 3. INTEGER Q
17  * 4. INTEGER_ARRAY special_integers
18  */
19  int mod=1e9+7;
20  long long getAns(vector<int> &nums, int s, int e,
    unordered_set<int> &set, int p, int q,
    vector<vector<long long>> &dp)
21  {
22      if(s==e) {
23          if(set.find(nums[s])!=set.end()) return
p%mod;
24          return q%mod;
25      }
26      if(dp[s][e]!=-1) return dp[s][e];
27      long long ans;
28      int spe=0;
29      for(int i=s;i<=e;i++)
30      {
31          if(set.find(nums[i])!=set.end()) spe++;
32      }
```

Line: 9 Col: 1

Test
Results

Custom
Input

Run
Code

Run
Tests

Submit

15s left



ALL



1

2

3

2. Get the Groups

As new students begin to arrive at college, each receives a unique ID number, 1 to n . Initially, the students do not know one another, and each has a different circle of friends. As the semester progresses, other groups of friends begin to form randomly.

There will be three arrays, each aligned by an index. The first array will contain a *queryType* which will be either *Friend* or *Total*. The next two arrays, *students1* and *students2*, will each contain a student ID. If the query type is *Friend*, the two students become friends. If the query type is *Total*, report the sum of the sizes of each group of friends for the two students.

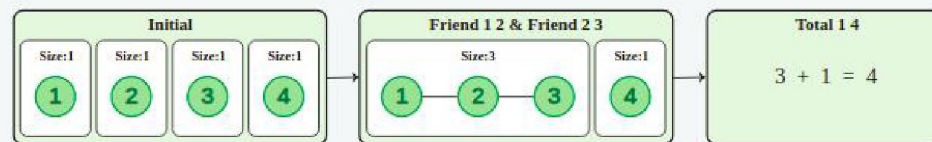
Example

$n = 4$

queryType = ['Friend', 'Friend', 'Total']

student1 = [1, 2, 1]

student2 = [2, 3, 4]



The queries are assembled, aligned by index:

Index	queryType	student1	student2
0	Friend	1	2

Language C++

Autocomplete Ready ⓘ

```
12  * The function is expected to return an
    INTEGER_ARRAY.
13  * The function accepts following parameters:
14  * 1. INTEGER n
15  * 2. STRING_ARRAY queryType
16  * 3. INTEGER_ARRAY students1
17  * 4. INTEGER_ARRAY students2
18  */
19  vector<int> par;
20  int find(int x)
21  {
22      if(par[x]<0) return x;
23      return find(par[x]);
24  }
25  void uni(int x, int y)
26  {
27      int parx=find(x);
28      int pary=find(y);
29      if(parx==pary) return;
30      if(abs(par[parx]) > abs(par[pary]))
31      {
32          par[parx]+=par[pary];
33          par[pary]=parx;
34      }
35      else {
36          par[pary]+=par[parx];
37          par[parx]=pary;
```

Line: 26 Col: 1

Test
ResultsCustom
InputRun
CodeRun
Tests

Submit

13s left

⌘

ALL

ⓘ

1

2

3

The queries are assembled, aligned by index:

Index	queryType	student1	student2
0	Friend	1	2
1	Friend	2	3
2	Total	1	4

Students will start as discrete groups $\{1\}$, $\{2\}$, $\{3\}$ and $\{4\}$. Students 1 and 2 become friends with the first query, as well as students 2 and 3 in the second. The new groups are $\{1, 2\}$, $\{2, 3\}$ and $\{4\}$ which simplifies to $\{1, 2, 3\}$ and $\{4\}$. In the third query, the number of friends for student 1 = 3 and student 4 = 1 for a Total = 4. Notice that student 3 is indirectly part of the circle of friends of student 1 because of student 2.

Function Description

Complete the function *getTheGroups* in the editor below. For a query of type Total with an index of j , the function must return an array of integers where the value at each index j denotes the answer.

getTheGroups has the following parameter(s):

- int n*: the number of students
- string queryType[q]*: an array of query type strings
- int student1[q]*: an array of student integer ID's
- int student2[q]*: an array of student integer ID's

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq q \leq 10^5$
- $1 \leq \text{students1}[i], \text{students2}[i] \leq n$
- queryTypes[i]* are in the set {'Friend', 'Total'}

Language C++
Autocomplete Ready ⓘ

```

12  * the function is expected to return an
    INTEGER_ARRAY.
13  * The function accepts following parameters:
14  * 1. INTEGER n
15  * 2. STRING_ARRAY queryType
16  * 3. INTEGER_ARRAY students1
17  * 4. INTEGER_ARRAY students2
18  */
19  vector<int> par;
20  int find(int x)
21  {
22      if(par[x]<0) return x;
23      return find(par[x]);
24  }
25  void uni(int x, int y)
26  {
27      int parx=find(x);
28      int pary=find(y);
29      if(parx==pary) return;
30      if(abs(par[parx]) > abs(par[pary]))
31      {
32          par[parx]+=par[pary];
33          par[pary]=parx;
34      }
35      else {
36          par[pary]+=par[parx];
37          par[parx]=pary;

```

Line: 26 Col: 1

Test Results
Custom Input
Run Code
Run Tests
Submit

10s left

⌘

ALL

ⓘ

1

2

3

- $1 \leq n \leq 10^3$
- $1 \leq q \leq 10^5$
- $1 \leq \text{students1}[i], \text{students2}[i] \leq n$
- $\text{queryTypes}[i]$ are in the set {'Friend', 'Total'}

► Input Format for Custom Testing

▼ Sample Case 0

Sample Input 0

STDIN	Function
3	→ n = 3
2	→ queryType[] size q = 2
Friend	→ query = ['Friend', 'Total']
Total	
2	→ students1[] size q = 2
1	→ students1 = [1, 2]
2	
2	→ students2[] size q = 2
2	→ students2 = [2, 3]
3	

Sample Output 0

3

Explanation 0

Initial

Friend 1 2

Total 2 3

Size:1

Size:1

Size:1

Size:2

Size:1

2 + 1 = 3

Language C++

Autocomplete Ready ⓘ

```

12  * the function is expected to return an
    INTEGER_ARRAY.
13  * The function accepts following parameters:
14  * 1. INTEGER n
15  * 2. STRING_ARRAY queryType
16  * 3. INTEGER_ARRAY students1
17  * 4. INTEGER_ARRAY students2
18  */
19  vector<int> par;
20  int find(int x)
21  {
22      if(par[x]<0) return x;
23      return find(par[x]);
24  }
25  void uni(int x, int y)
26  {
27      int parx=find(x);
28      int pary=find(y);
29      if(parx==pary) return;
30      if(abs(par[parx]) > abs(par[pary]))
31      {
32          par[parx]+=par[pary];
33          par[pary]=parx;
34      }
35      else {
36          par[pary]+=par[parx];
37          par[parx]=pary;

```

Line: 26 Col: 1

Test

Custom

Run Code

Run Tests

Submit

9s left



ALL



1

2

3

Friend → query = ['Friend', 'Total']

Total

2 → students1[] size q = 2

1 → students1 = [1, 2]

2

2 → students2[] size q = 2

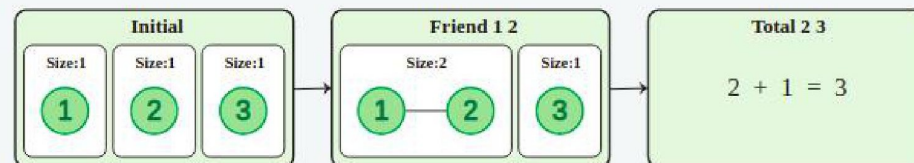
2 → students2 = [2, 3]

3

Sample Output 0

3

Explanation 0



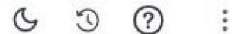
There are $n = 3$ students labeled with the IDs $\{1, 2, 3\}$. Perform the following $q = 2$ queries:

0. *Friend 1 2*: Students 1 and 2 become friends.
1. *Total 2 3*: Find the total number of students in the collective friend groups of students 2 and 3. Student 2 is in a group with two friends (i.e., $\{1, 2\}$) and student 3 is in a group with one friend (i.e., $\{3\}$), so store $2 + 1 = 3$ in index 0 of the return array.

After performing all the queries, return the array $[3]$ as the answer.

► Sample Case 1

Language C++



✓ Autocomplete Ready ⓘ

```

12  * The function is expected to return an
    INTEGER_ARRAY.
13  * The function accepts following parameters:
14  * 1. INTEGER n
15  * 2. STRING_ARRAY queryType
16  * 3. INTEGER_ARRAY students1
17  * 4. INTEGER_ARRAY students2
18  */
19  vector<int> par;
20  int find(int x)
21  {
22      if(par[x]<0) return x;
23      return find(par[x]);
24  }
25  void uni(int x, int y)
26  {
27      int parx=find(x);
28      int pary=find(y);
29      if(parx==pary) return;
30      if(abs(par[parx]) > abs(par[pary]))
31      {
32          par[parx]+=par[pary];
33          par[pary]=parx;
34      }
35      else {
36          par[pary]+=par[parx];
37          par[parx]=pary;
  
```

Line: 26 Col: 1

Test
Results

Custom
Input

Run
Code

Run
Tests

Submit

36m left



ALL



1

2

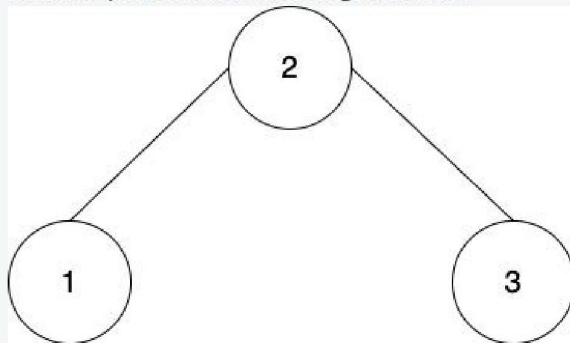
3

3. Special Nodes

Given a tree with N nodes, a node is called special if it is an end-point of any of the diameters of the tree. For all the nodes of the tree, find if it is special or not. Thus, return a binary array, where the i^{th} value of the array will be 1, in case the i^{th} node is special. Else, the i^{th} value of the array will be 0.

Note: The diameter of a tree is defined as the number of edges in the longest path of the tree.

For example, consider the tree given below:



We can see that this tree has only one diameter, which is the unique path between nodes 1 and 3. The length of the diameter is 2.

The end-points of the diameter are 1 and 3. Hence, nodes 1 and 3 are considered special, while node 2 is not considered a special node of the tree.

Language C++

Autocomplete Ready ⓘ

```
16 */
17
18 /*
19  * For the unweighted graph, <name>:
20  *
21  * 1. The number of nodes is <name>_nodes.
22  * 2. The number of edges is <name>_edges.
23  * 3. An edge exists between <name>_from[i] and
24  *    <name>_to[i].
25  */
26
27 vector<int> isSpecial(int n, vector<int> tree_from,
28 vector<int> tree_to) {
29     unordered_map<int, vector<int>> adj;
30     for(int i=0; i<tree_from.size(); i++)
31     {
32         adj[tree_from[i]].push_back(tree_to[i]);
33         adj[tree_to[i]].push_back(tree_from[i]);
34     }
35     int diameter = 0;
36     for(int i=0; i<adj.size(); i++)
37     {
38         diameter = max(diameter, 1 + max(adj[i].size(), adj[adj[i][0]].size()));
39     }
40 }
```

Line: 37 Col: 24

Test
ResultsCustom
InputRun
CodeRun
Tests

Submit

36m left



ALL



1

2

3

We can see that this tree has only one diameter, which is the unique path between nodes 1 and 3. The length of the diameter is 2.

The end-points of the diameter are 1 and 3. Hence, nodes 1 and 3 are considered special, while node 2 is not considered a special node of the tree.

Function Description

Complete the function *isSpecial* in the editor below.

isSpecial has the following parameter(s):

tree_nodes: The number of nodes in the tree.

tree_from: The nodes from which the edge is incident onto the other node

tree_to: The nodes on which the edge is incident

Return

int[n]: an integer array of size N, where the i^{th} value of the array is 1, in case the i^{th} node is special. Else, the i^{th} value will be 0.

Constraints

- $1 \leq \text{tree_nodes} \leq 10^5$
- $|\text{tree_from}| = |\text{tree_to}| = \text{tree_nodes} - 1$
- $1 \leq \text{tree_from}_i, \text{tree_to}_i \leq \text{tree_nodes}$

► Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

STDIN FUNCTION

Language C++

Autocomplete Ready ⓘ

```
16  */
17
18  /*
19  * For the unweighted graph, <name>:
20  *
21  * 1. The number of nodes is <name>_nodes.
22  * 2. The number of edges is <name>_edges.
23  * 3. An edge exists between <name>_from[i] and
   <name>_to[i].
24  *
25  */
26
27  vector<int> isSpecial(int n, vector<int> tree_from,
   vector<int> tree_to) {
28      unordered_map<int, vector<int>> adj;
29      for(int i=0; i<tree_from.size(); i++)
30      {
31          adj[tree_from[i]].push_back(tree_to[i]);
32          adj[tree_to[i]].push_back(tree_from[i]);
33      }
34      int diameter=0;
35      for(int i=0; i<tree_from.size(); i++)
36      {
37          diameter = max(diameter,
38                          max(initializer_list<Tp> l) -> _Tp
39                          (adj[tree_from[i]].size(),
40                          adj[tree_to[i]].size()));
41      }
42  }
```

Line: 37 Col: 24

Test
Results

Custom
Input

Run
Code

Run
Tests

Submit

36m left

► Input Format For Custom Testing

▼ Sample Case 0

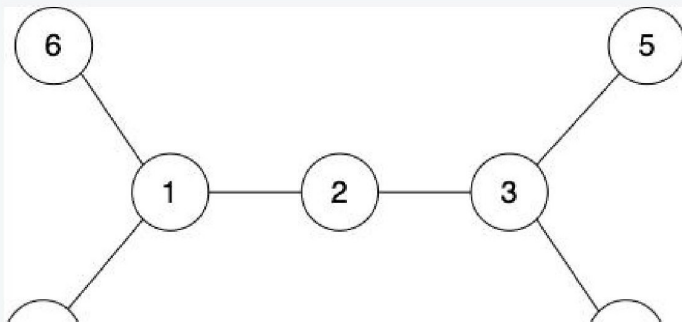
Sample Input For Custom Testing

STDIN	FUNCTION
7 6	-> N = 7 and N-1 = 6
1 2	-> edges = [1 2, 2 3, 3 4, 3 5, 3 5, 1 6, 1 7]
2 3	
3 4	
3 5	
1 6	
1 7	

Sample Output

0 0 0 1 1 1 1

Explanation



Language C++

Autocomplete Ready ⓘ

```
16 */
17
18 /*
19  * For the unweighted graph, <name>:
20  *
21  * 1. The number of nodes is <name>_nodes.
22  * 2. The number of edges is <name>_edges.
23  * 3. An edge exists between <name>_from[i] and
   <name>_to[i].
24  *
25  */
26
27 vector<int> isSpecial(int n, vector<int> tree_from,
   vector<int> tree_to) {
28     unordered_map<int, vector<int>> adj;
29     for(int i=0;i<tree_from.size();i++)
30     {
31         adj[tree_from[i]].push_back(tree_to[i]);
32         adj[tree_to[i]].push_back(tree_from[i]);
33     }
34     int diameter=0;
35     for(int i=0;i<adj.size();i++)
36     {
37         diameter = max(diameter, isSpecial(adj[i], tree_from, tree_to));
38     }
39 }
40
```

Line: 37 Col: 24

Test
ResultsCustom
InputRun
CodeRun
Tests

Submit

36m left



ALL



1

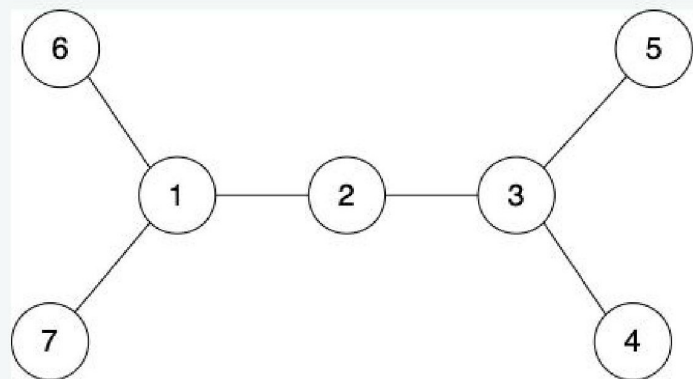
2

3

```
1 2    -> edges = [ 1 2, 2 3, 3 4, 3 5, 3 5, 1 6, 1 7 ]
2 3
3 4
3 5
1 6
1 7
```

Sample Output

```
0 0 0 1 1 1 1
```

Explanation

We have four diameters in this tree, which are the unique paths between the nodes (6,5) (6,4) (7,5) and (7,4). The special nodes are nodes 4,5,6,7 (since they are the end-points of at least one of the diameters of the given tree).

Sample Case 1

Language C++

Autocomplete Ready ⓘ

```
16  */
17
18  /*
19   * For the unweighted graph, <name>:
20   *
21   * 1. The number of nodes is <name>_nodes.
22   * 2. The number of edges is <name>_edges.
23   * 3. An edge exists between <name>_from[i] and
24   *    <name>_to[i].
25   */
26
27  vector<int> isSpecial(int n, vector<int> tree_from,
28                      vector<int> tree_to) {
29      unordered_map<int, vector<int>> adj;
30      for(int i=0;i<tree_from.size();i++)
31      {
32          adj[tree_from[i]].push_back(tree_to[i]);
33          adj[tree_to[i]].push_back(tree_from[i]);
34      }
35      int diameter=0;
36      for(int i=0;i<adj.size();i++)
37      {
38          diameter = max(diameter, isSpecial(i, adj[i], adj[i]));
39      }
40  }
```

Line: 37 Col: 24

Test

Custom

Run

Run

Submit